

Architecture Patterns for AI Applications

Module 8 — Business AI in Practice · Notebook 29

Prof. Dr. Christoph Weisser

HSBI

Summer Semester 2026

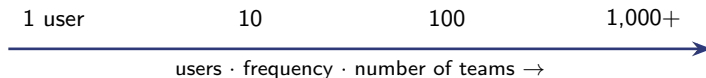
Contents

- 1 Traffic
- 2 Patterns
- 3 Pipeline
- 4 Choosing
- 5 Course map

Outline

- 1 Traffic
- 2 Patterns
- 3 Pipeline
- 4 Choosing
- 5 Course map

Architecture is the answer to a *traffic* question



Ask three things before you choose

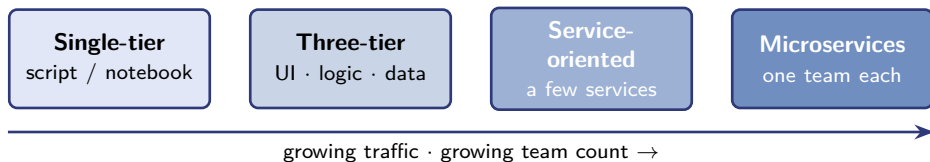
- **How many users**, and growing how fast?
- **Who invokes it** — a human, a schedule, or another service?
- **How many teams** will own and ship pieces of it?

Architecture is not sophistication — it is matching *structure* to *load*.

Outline

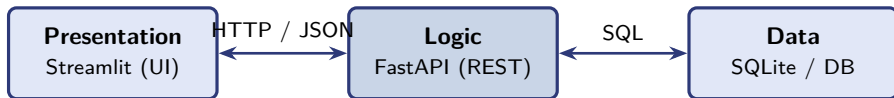
- 1 Traffic
- 2 Patterns
- 3 Pipeline
- 4 Choosing
- 5 Course map

Four patterns of growing complexity



Read left to right as *growing load*, not “primitive to advanced”. The simplest workable choice wins.

Three-tier — the workhorse for interactive AI features



- Each tier can change independently; the **REST contract** is the boundary.
- The API is reusable — a second UI, a CLI, or another team can call it.
- *You build exactly this* in **Notebook 34 (POC 2)**.

The over-engineering trap

Microservices for a 5-user pilot

Microservices became fashionable around 2015; many teams adopted them *before* they had the team size or traffic to justify the operational cost. The right unit of a service is a *capability* — not every Python module.

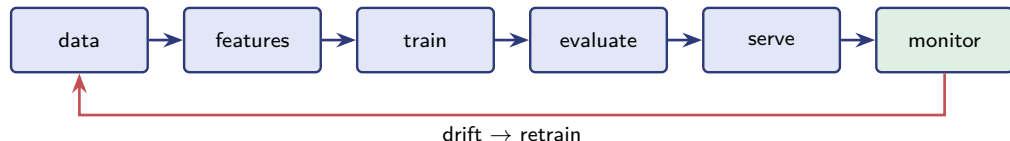
The rule

Pick the architecture you'll have *outgrown in 12 months*, not the one you'll *need in 36*.
Premature complexity kills more projects than migrations do.

Outline

- 1 Traffic
- 2 Patterns
- 3 Pipeline**
- 4 Choosing
- 5 Course map

The cross-cutting ML / AI pipeline



What pure-software patterns don't capture

- **Training data is part of the artefact** — different snapshot, different product.
- **Drift exists** — 92 % in March is not 92 % in October.
- **The feedback loop has economic cost** — each prediction nudges the next training batch.

Outline

- 1 Traffic
- 2 Patterns
- 3 Pipeline
- 4 Choosing**
- 5 Course map

Choosing the right size

Smells that you've outgrown your current shape

- “Can we run this nightly?” → add scheduling (single-tier job).
- A non-developer needs to use it → add a UI tier (three-tier).
- A second team wants to build on it → expose it as a service.

Two opposite mistakes

Over-engineering (microservices for a pilot) and **under-engineering** (a 200-user system held together by one notebook).

Outline

- 1 Traffic
- 2 Patterns
- 3 Pipeline
- 4 Choosing
- 5 Course map

How the course maps to these patterns

Pattern	Where it appears in the course
Single-tier (script / notebook)	Every notebook NB 1–27, by default
Scheduled single-tier	NB 24 — scheduling & orchestration
Three-tier with AI in the middle	NB 34 (POC 2) and NB 27 capstone
Service-oriented	<code>llm_providers.py</code> — pluggable providers
Microservices	<i>patterns only</i> — too big for a teaching course
ML pipeline	NB 15–17 + NB 24 — train / evaluate / serve

You *build* the three-tier and ML-pipeline shapes in **Notebook 34**; the rest you meet as patterns here.